

Documento Técnico IAW de Proyecto Intermodular LogiTrans S.A.



LogiTrans S.A

AI generated

David Felipe Manosalva Niño

Tutor: Juan Sánchez González

C.F.G.S. Administración de Sistemas Informáticos en Red

IES Albarregas 2025/26

Fecha entrega: 21/03/26

ÍNDICE

Infraestructura Web	2
1. Visión General de la Infraestructura Web	2
1.1 Requisitos de Hardware	2
1.2 Stack de Software	2
2. Visión General de la Infraestructura Web	3
2.1 Función en la infraestructura	3
2.2 Algoritmo de balanceo	3
2.3 Comprobación de estado	3
2.4 Panel de estadísticas	4
3. Servidores Web (Apache)	4
3.1 Función en la infraestructura	4
3.2 Configuración básica	4
3.3 Contenido Estático frente a Dinámico	5
4. Servidor de Aplicación (PHP-FPM)	5
4.1 Función y justificación de instancia única	5
4.2 Configuración	5
5. Base de datos (MariaDB)	6
5.1 Tecnología y Arquitectura	6
5.2 Modelo de Datos	6
5.3 Usuarios y Permisos	6
Aplicación Web Pública	7
6. Plataforma Web LogiTrans	7
6.1 Estructura de la aplicación	7
6.2 Servicios disponibles en la Web	8
6.3 Flujo de solicitud de servicio	8
6.4 Seguridad de la aplicación web	8
Continuidad del Servicio	9
7. Alta disponibilidad y tolerancia a fallos	9
7.1 Puntos Críticos del Sistema	9
7.2 Mecanismos implementados	9
7.3 Escenarios de fallo	10
8. Monitorización y mantenimiento	10
8.1 Herramientas de monitorización	10
8.2 Tareas de Mantenimiento periodico	11
9. Esquema	11

Infraestructura Web

1. Visión General de la Infraestructura Web

La infraestructura web de LogiTrans se despliega en el Docker host (192.168.50.10) mediante Docker Compose. Todos los servicios corren como contenedores aislados conectados por redes Docker internas. El tráfico público entra exclusivamente por el puerto 80 del Docker host, que lo mapea al balanceador HAProxy.

1.1 Requisitos de Hardware

Recurso	Especificación
Host Físico CPU	8 núcleos
Host Físico RAM	8 GB mínimo (16 GB recomendado)
Host Físico Almacenamiento	120 GB SSD para imágenes de VM y volúmenes Docker
Docker Host CPU	2 núcleos virtuales
Docker Host RAM	2 GB — los contenedores consumen aprox 1.5 GB
Docker Host disco	40 GB — SO Ubuntu + imágenes Docker + volúmenes

1.2 Stack de Software

Recurso	Versión	Especificación
Ubuntu Server	22.04 LTS	8 núcleos
Docker	28.5.1	8 GB mínimo (16 GB recomendado)
Docker Compose	v2.40.2	120 GB SSD para imágenes de VM y volúmenes Docker
HAProxy	2.9	2 núcleos virtuales

Apache HTTP server	2.4	2 GB — los contenedores consumen aprox 1.5 GB
PHP-FPM	8.2	40 GB — SO Ubuntu + imágenes Docker + volúmenes
MariaDB	11.2	Sistema Gestor de base de datos relacionales

2. Visión General de la Infraestructura Web

2.1 Función en la infraestructura

HAProxy es el primer contenedor que recibe el tráfico web desde el Docker host. Tiene doble interfaz de red: IF1 en red_dmz donde recibe el tráfico mapeado desde el puerto 80 del host, e IF2 en red_web desde donde distribuye las peticiones entre los servidores Apache. No tiene visibilidad ni acceso a las capas PHP ni MariaDB.

2.2 Algoritmo de balanceo

Se utiliza Round-Robin, que distribuye las peticiones de forma secuencial y equitativa entre los dos Apache. Es adecuado porque ambos nodos tienen la misma capacidad y no almacenan el estado del usuario. Las sesiones de usuario residen en el servidor PHP, que es una instancia única.

2.3 Comprobación de estado

HAProxy verifica periódicamente la disponibilidad de cada Apache realizando una petición GET a la raíz del servidor. Si un nodo no responde, es excluido automáticamente del pool y el 100% del tráfico se redirige al disponible. Cuando se recupera, se reincorpora sin intervención manual.

2.4 Panel de estadísticas

HAProxy expone una interfaz web en el puerto 8080 del Docker host accesible solo desde la LAN corporativa. Muestra en tiempo real el estado de cada Apache, conexiones activas, tasa de peticiones y errores. El acceso está protegido con usuario y contraseña.

3. Servidores Web (Apache)

3.1 Función en la infraestructura

Los dos servidores Apache tienen doble interfaz de red. A través de su IF1 en red_web reciben tráfico de HAProxy. A través de su IF2 en red_php delegan las peticiones dinámicas al servidor PHP. Sirven directamente los recursos estáticos (HTML, CSS, imágenes) sin invocar PHP ni la base de datos.

3.2 Configuración básica

Recurso	Especificación
Imagen	httpd:2.4
Módulos Habilitados	mod_proxy, mod_proxy_fcgi
Punto de Escucha	80
DocumentRoot	/var/www/html (bind mount desde el Docker host)
Peticiones PHP	Reenviadas a logitrans-php:9000 mediante FastCGI
HTTPS	Pendiente. Se planea delegar el certificado SSL en HAProxy.

3.3 Contenido Estático frente a Dinámico

Recurso	Especificación	Procesado por
Estático	HTML, CSS, Imágenes, JS	Apache directamente. Sin PHP ni base de datos
Dinámico	login.php, registro.php etc.	Apache reenvía a PHP-FPM que procesa y devuelve el HTML

4. Servidor de Aplicación (PHP-FPM)

4.1 Función y justificación de instancia única

El servidor PHP ejecuta toda la lógica de la aplicación web: autenticación de usuarios, consultas a la base de datos, validación de formularios y generación de páginas dinámicas. Tiene doble interfaz: IF1 en red_php donde recibe peticiones de los Apache, e IF2 en red_bd desde donde consulta MariaDB.

Se usa una única instancia PHP para resolver el problema de las sesiones. Con dos instancias PHP independientes, el usuario podrá autenticarse en una y perder la sesión si la siguiente petición llegará al Apache conectado con la otra instancia. Con una única instancia central todas las sesiones existen en el mismo lugar siempre.

4.2 Configuración

Parámetro	Valor
Imagen Base	php:8.2-fpm
Extensiones	mysqli, pdo, pdo_mysql
Puerto de escucha	9000 (FastCGI)
Sesiones	Almacenadas el volumen Docker persistente php sessions
Conexiones a BD	PDO con consultas preparadas.

5. Base de datos (MariaDB)

5.1 Tecnología y Arquitectura

MariaDB 11.2 es el componente más aislado de la infraestructura: reside únicamente en red_bd, sin gateway y sin otras interfaces de red. Solo responde peticiones del servidor PHP. Los datos persisten en un volumen Docker nombrado independiente del ciclo de vida del contenedor.

5.2 Modelo de Datos

Parámetro	Valor	
Estáticas	DEPARTAMENTOS, PUESTOS_TRABAJO	Datos de referencia. 8 departamentos y 23 puestos.
Semiestáticas	EMPLEADOS, VEHÍCULOS, EQUIPOS	48 empleados del AD, 8 vehículos, 10 equipos.
Dinámicas	CLIENTES, MERCANCIA, ENVIOS, DETALLE_ENVIO, MOVIMIENTO_BODEGA	Crecen con la operativa diaria.
Solicitudes	SOLICITUDES	Peticiones de servicio de clientes desde la web pública. Estado: PENDIENTE > REVISANDO > ACEPTADA/RECHAZADA.

5.3 Usuarios y Permisos

Parámetro	Valor	
root	Superusuario completo	Sólo administración directa. Nunca desde la aplicación.
logitrans_user	Todos los permisos sobre logitrans_db	Usuario que usa PHP para conectarse a la BD.
web_publica	SELECT e INSERT limitados	Pendiente. Para la parte pública sin acceso a datos sensibles.

Aplicación Web Pública

6. Plataforma Web LogiTrans

La plataforma web pública de LogiTrans es accesible desde internet a través del puerto 80 del Docker host. Está desarrollada en PHP 8.2 con Bootstrap 5 como framework CSS y se conecta a la base de datos MariaDB mediante PDO con consultas preparadas.

6.1 Estructura de la aplicación

Página	Estado	Descripción
index.php	Implementado (puede mejorar)	Página principal. Presenta la empresa, misión, visión, valores, los cuatro servicios disponibles y sección de contacto.
registro.php	Implementado (puede mejorar)	Formulario de alta de clientes. Valida el teléfono, comprueba unicidad del email y almacena la contraseña con hash seguro.
login.php	Implementado (puede mejorar)	Autenticación de clientes. Verifica credenciales con password_verify y gestiona la sesión de usuario.
dashboard.php	Implementado (puede mejorar)	Panel del cliente autenticado. Muestra las solicitudes enviadas con su estado actual.
solicitar.php	Implementado (puede mejorar)	Formulario de solicitud de servicios. Muestra campos distintos según el servicio seleccionado e inserta en la tabla SOLICITUDES.
logout.php	Implementado (puede mejorar)	Destruye la sesión activa y redirige al inicio.
header.php	Implementado (puede mejorar)	Componente de navegación reutilizable. Muestra opciones distintas según si el usuario está autenticado o no.

6.2 Servicios disponibles en la Web

Página	Descripción
Transporte de mercancías	Traslado de mercancías entre cualquier punto de España. Flota propia con capacidad de hasta 20.000 kg.
Almacenamiento en bodega	Almacenamiento temporal o prolongado en nave propia en Mérida.
Transporte Urgente	Entrega prioritaria en el menor tiempo posible. Ideal para envíos de última hora.
Logística Integral	Combinación de almacenamiento y transporte en un único servicio.

6.3 Flujo de solicitud de servicio

1. El cliente accede a la web pública y visualiza los servicios disponibles.
2. Si no tiene cuenta, se registra mediante el formulario de registro.
3. Inicia sesión con sus credenciales.
4. Selecciona el servicio deseado y rellena el formulario de solicitud.
5. La solicitud se guarda en la tabla SOLICITUDES con estado PENDIENTE.
6. El cliente puede consultar el estado de sus solicitudes desde su panel.
7. Un empleado de LogiTrans gestiona la solicitud desde el sistema interno y la convierte en envío real.

6.4 Seguridad de la aplicación web

- Contraseñas almacenadas con hash seguro (password_hash de PHP). Nunca en texto plano.
- Consultas preparadas con PDO en todas las operaciones de base de datos. Elimina el riesgo de inyección SQL.
- Salidas HTML escapadas con htmlspecialchars() para prevenir XSS.
- Regeneración del ID de sesión tras el login para prevenir sesión de sesión.
- Validación de campos en servidor (PHP) además de la validación en cliente (HTML5).

Continuidad del Servicio

7. Alta disponibilidad y tolerancia a fallos

7.1 Puntos Críticos del Sistema

Componente	Resiliencia	Análisis
Apache x2	Alta — redundante	Activo-activo. Si uno cae, HAProxy detecta el fallo y redirige todo el tráfico al nodo disponible.
HAProxy	Media — SPOF	Su caída deja el servicio inaccesible. Mitigado con reinicio automático Docker.
PHP-FPM	Media — SPOF	Su caída genera errores en páginas dinámicas. Reinicio automático Docker. Sesiones conservadas en volumen.
MariaDB	Media — SPOF	Su caída hace inoperativa la aplicación. Datos protegidos por volumen y backups.
Docker Host	SPOF critico	Si la VM cae, toda la infraestructura web cae. Depende de la estabilidad del hipervisor.

7.2 Mecanismos implementados

- Dos Apache en activo-activo con detección automática de fallos por HAProxy.
- Reinicio automático de todos los contenedores ante caídas inesperadas (restart: unless-stopped).
- Volumen persistente de base de datos: los datos sobreviven a reinicios del contenedor.
- Volumen persistente de sesiones PHP: las sesiones sobreviven a reinicios del servidor PHP.
- Backups regulares para recuperación ante pérdida total.

7.3 Escenarios de fallo

Componente	Impacto	Respuesta automática
Caída de un Apache	Ninguno visible para el usuario	HAProxy detecta el fallo y redirige todo el tráfico al nodo disponible. Docker reinicia el caído.
Caída de HAProxy	Servicio inaccesible	Docker reinicia el contenedor automáticamente en segundos.
Caída de PHP	Errores en páginas dinámicas	Docker reinicia el contenedor. Las sesiones se conservan en el volumen.
Caída de MariaDB	Aplicación inoperativa	Docker reinicia el contenedor. Los datos no se pierden.
Caída del Docker host	Toda la infraestructura web cae	Reinicio manual de la VM. Docker levanta todos los contenedores automáticamente al arrancar.

8. Monitorización y mantenimiento

8.1 Herramientas de monitorización

Herramienta	Que monitoriza	Cómo acceder
Panel HAProxy Stats	Estado de los Apache, conexiones, tasa de peticiones y errores.	http://192.168.50.10:8080/stats — solo desde la LAN.
docker stats	CPU, memoria, red y disco de cada contenedor en tiempo real.	Ejecutado en el Docker host.
docker ps	Estado y tiempo de actividad de cada contenedor.	Ejecutado en el Docker host.
docker logs	Logs de Apache, PHP y MariaDB.	<code>docker logs [nombre_contenedor]</code>

8.2 Tareas de Mantenimiento periodico

Frecuencia	Que monitoriza
Diario	Revisar estado de contenedores y errores recientes en logs.
Semanal	Verificar que el último backup se generó correctamente.
Mensual	Actualizar imágenes Docker base y reconstruir contenedores.
Mensual	Limpiar imágenes Docker antiguas y capas no utilizadas.

9. Esquema

